



Personal Finance Manager

Aplicación web para gestión personal de finanzas.

Autor: Abraham Quintana Micó

Introducción

- **Propósito:** facilitar registro, clasificación, análisis y control de ingresos y gastos.
- **Público objetivo:** usuarios particulares y pequeñas empresas.
- **Enfoque UI/UX:** claridad, visualización y facilidad de uso.

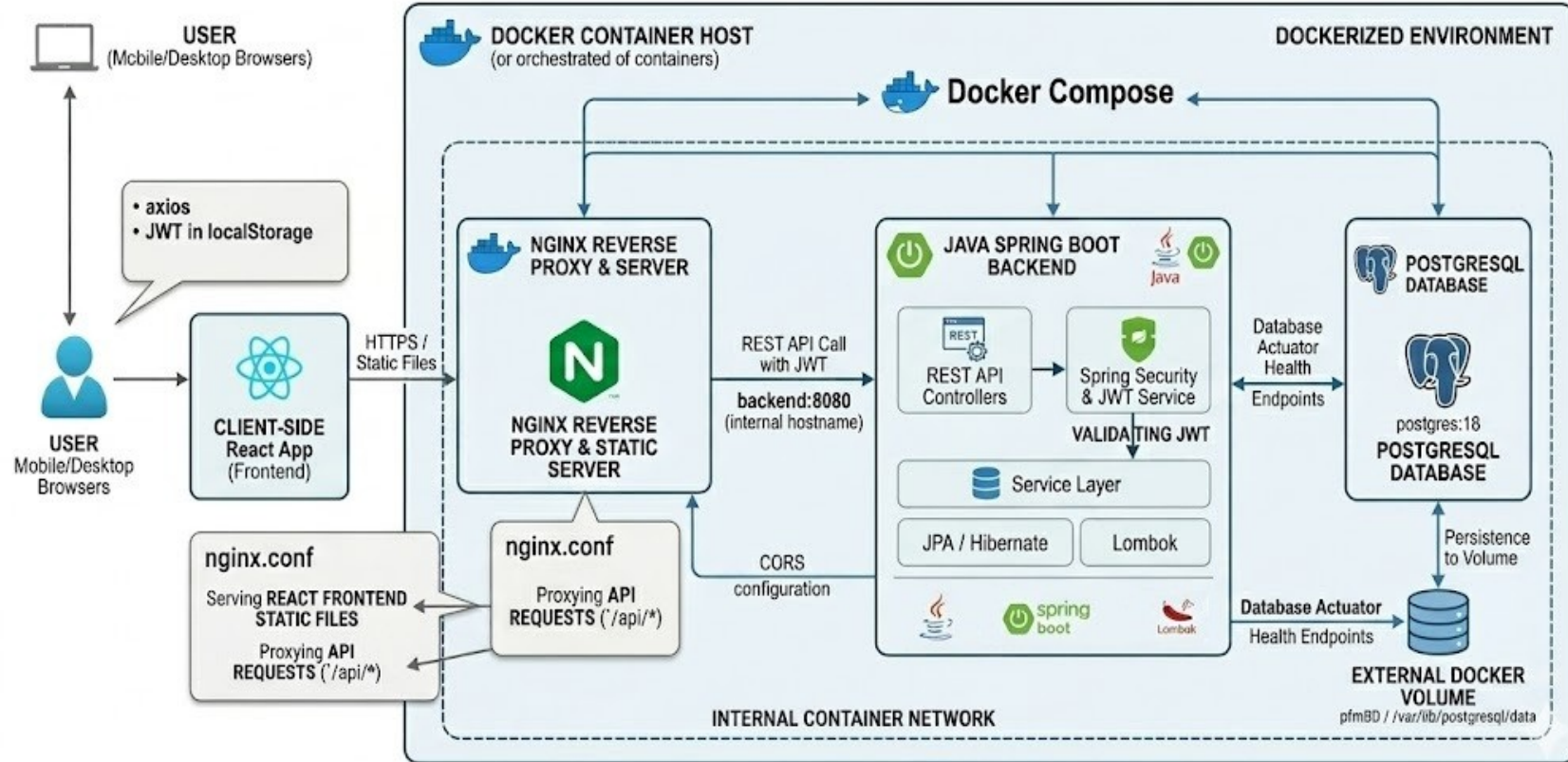
Objetivo general y específicos

- **Objetivo general:** herramienta completa para gestionar finanzas personales.
- **Objetivos específicos:**
 - Autenticación segura
 - Gestión de transacciones y categorías,
 - Automatización de transacciones recurrentes
 - Indicadores financieros
 - Visualizaciones de transacciones y estado financiero

Tecnologías principales

- **Backend:** Spring Boot (Java 21).
- **Frontend:** React (create-react-app).
- **Base de datos:** PostgreSQL.
- **Contenerización:** Docker y Docker Compose.
- **Gráficos:** Chart.js y react-chartjs-2.

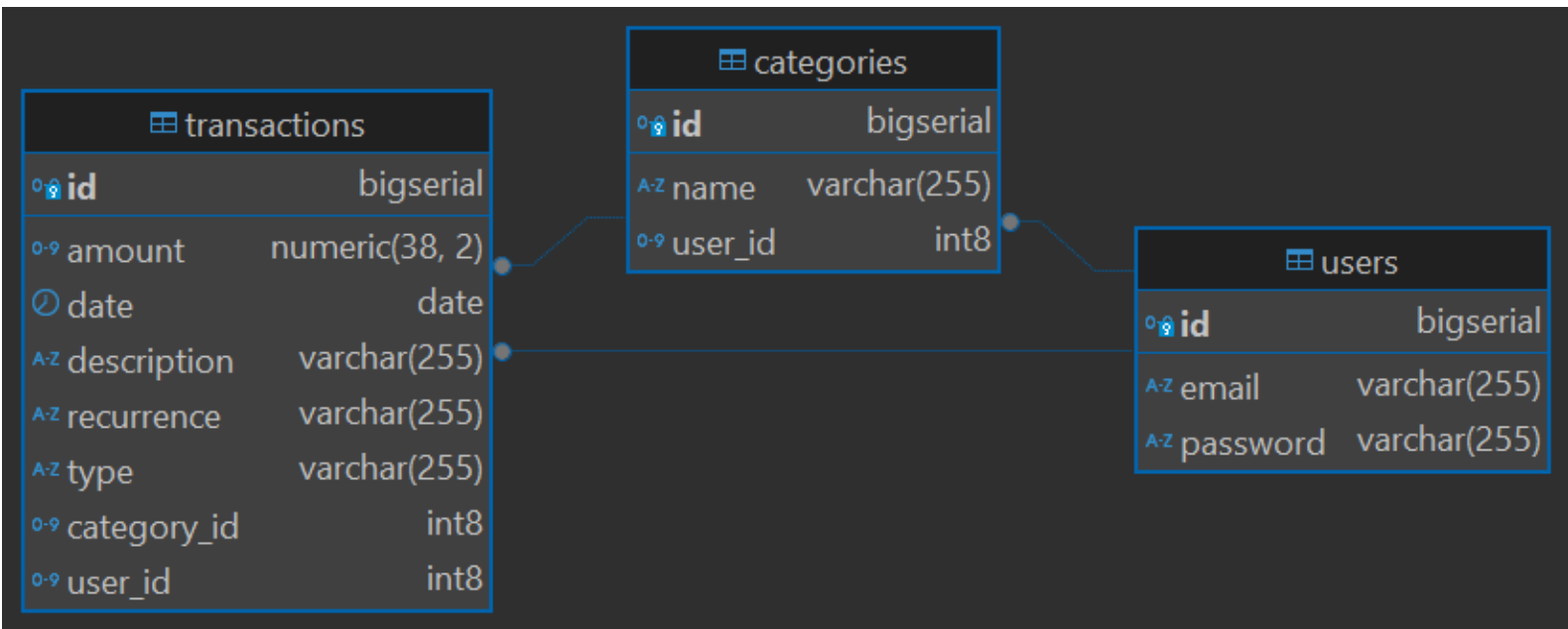
PROJECT ARCHITECTURE - PERSONAL FINANCE MANAGER



Arquitectura
general

Servidor o Back-End

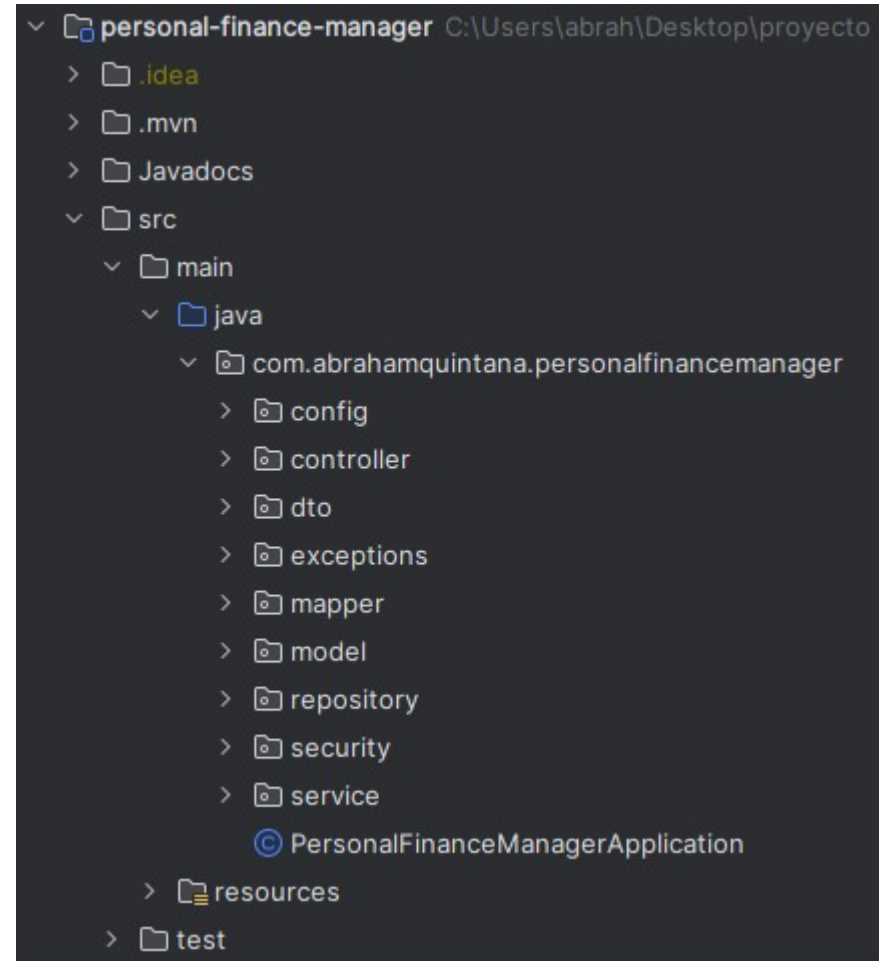




- Entidades principales: User, Category, Transaction.
- Restricciones unicidad:
 - Usuario → Email.
 - Categoría → User y Nombre.
- Tipos enumerados:
 - Transaction Type (INCOME/EXPENSE)
 - Recurrence (NO/MONTHLY/YEARLY).

Backend estructura

- Arquitectura limpia con distribución por paquetes:
 - Config
 - Controller
 - Dto
 - Exceptions
 - Mapper
 - Model
 - Repository
 - Security
 - Service



Seguridad y autenticación



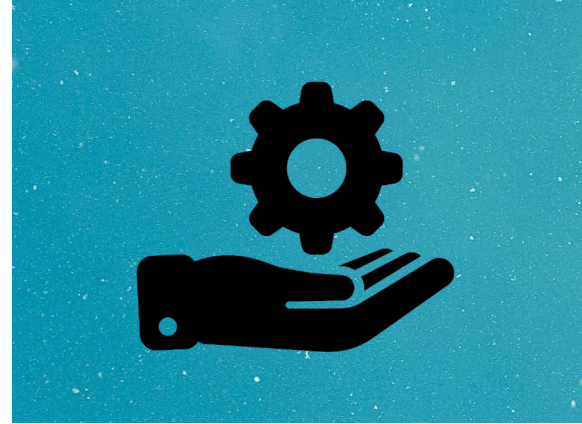
Esquema obtenido de OpenWebinars (<https://openwebinars.net/blog/que-es-json-web-token-y-como-funciona>)

- Autenticación basada en JWT.
- Almacenamiento seguro de contraseñas (hash).
- Cadena de filtros con:
 - Rutas permitidas sin autenticación
 - Rutas que requieren token
 - Configuración CORS para permitir acceso desde el cliente local y desde la red interna en Docker.

UserFinancialStateService

Uso y responsabilidades

- El servicio tiene un método público que se llama solo desde el UserController `"/api/user/state"`
- Se encarga de gestionar y devolver el estado global de transacciones del usuario mediante **UserFinancialStateDto**.
 - 1) **Genera y actualiza las transacciones recurrentes del usuario.**
 - 2) **Prepara los datos del UserFinancialStateDto del mes con:**
 - MonthInitialBalance (Balance disponible inicial del mes)
 - CurrentBalance (Balance disponible actual)
 - Income (ingresos realizados hasta la fecha)
 - Expenses (gastos realizados hasta la fecha)
 - DailyAvailable (dinero restante disponible por día)
 - TodayAvailable (dinero disponible para hoy)
 - Past (transacciones ya realizadas este mes)
 - Future (transacciones por realizar este mes)



Lógica de recurrencia

```
/**
 * Generates new transactions from recurring rules if their execution date is due
 * And marks the old transactions with recurrence = No
 */
1 usage  Abraham Quintana +1
private void generateAndUpdateRecurringTransactionsIfNeeded(User user) {
    LocalDate today = LocalDate.now();
    List<Transaction> allTransactions = transactionRepository.findByUserAndRecurrenceNot(user, RecurrenceType.NO);
    List<Transaction> transactionsToSave = new ArrayList<>();

    allTransactions.forEach( transaction -> {
        //define next transaction date
        if(transaction.getDate().isBefore(today)) {
            LocalDate recurrenceDate = getRecurrenceDate(transaction);

            // Generate next transaction based on the recurrence rule
            Transaction nextTransaction = getNextTransaction(transaction, recurrenceDate);

            //mark old transaction as NO recurrence
            transaction.setRecurrence(RecurrenceType.NO);

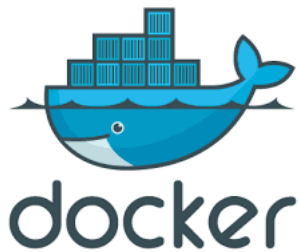
            //add old transaction to the list of transactions to be saved
            transactionsToSave.add(nextTransaction);
        }
    });

    transactionRepository.saveAll(transactionsToSave);
}
```

Flujo

- 1) Obtiene todas las transacciones del usuario que tienen recurrencia distinta de "NO".
- 2) En aquellas cuya fecha es anterior a hoy:
 - 1) Calcula la fecha de la siguiente transacción dependiendo del valor de "recurrence" que tenga (MONTHLY, YEARLY).
 - 2) Crea una nueva transacción opero con la fecha calculada en el paso anterior.
 - 3) Cambia el tipo de recurrencia a "NO", para que no vuelva a ser tenida en cuenta la siguiente vez que se llame al método.
 - 4) Agrupa todas las nuevas transacciones en una lista.
 - 5) Persiste la lista de transacciones en una única llamada a BBDD.

Base de datos y Docker



Instalación de DockerDesktop para su uso desde Windows. Permite crear volúmenes donde persistir la información entre reinicios.



Es el repositorio de imágenes de docker en este caso. Para descargar la imagen de docker vale con ejecutar:
docker pull postgres



Base de datos relacional de código abierto, robusta, altamente extensible y compatible con estándares SQL avanzados.

Cliente o Font-End



**Cliente basado
en NodeJs +
Framework
ReactJs**

Node.js

Constituye un entorno de ejecución veloz basado en eventos, ideal para aplicaciones escalables en tiempo real utilizando JavaScript en el servidor.

React es un Framework que utiliza el concepto de SPA (Single Page Application) carga una única página HTML y actualiza el contenido dinámicamente sin recargar el navegador, mediante componentes.

Frontend Estructura

- Arquitectura limpia con distribución por directorios:
 - Assets
 - Components
 - Context
 - Router
 - Services
 - Styles
 - Utils
 - (...ficheros base)



```
▼ src
  > assets
  > components
  > context
  > router
  > services
  > styles
  > utils
  JS App.js
  JS App.test.js
  JS index.js
  JS reportWebVitals.js
  JS setupTests.js
  .gitignore
  Dockerfile
  nginx.conf
  {} package-lock.json
  {} package.json
  ⓘ README.md
```

Componentes principales

- **Header y Footer** payload que aparece en toda la aplicación.
 - **Header** dinámico con botones de Login y registro o Logout.
 - **Footer** con información del desarrollador y link al Github, además de versión de la app.
- **Home** (página de inicio): Mensaje de bienvenida dinámico e indicaciones para Login o Registro
- **Registry** → registerUser → token → AuthContext.login → redirección a Profile.
- **Login** → loginUser → token → AuthContext.login → redirección a Profile.
- **UserNav**: Componente que se usa dentro de los componentes protegidos y el Home(con usuario autenticado) para permitir la navegación entre los mismos.
- **Profile** (protegida): Resumen del perfil y estado actual de finanzas del usuario Tarjetas con “Saldo actual, Ingresos este mes, Gastos este mes, Disponible diario” y un listado con las 5 últimas transacciones registradas.
- **Category** (protegida): Creación de categorías del usuario.
- **Transaction** (protegida): Creación y listado de transacciones. (La creación de transacciones requiere categorías, permite eliminar transacciones)
- **Statistics** (protegida): Gráficas con (evolución del saldo + gastos e ingresos pasados y futuros categoría) y Listados de (transacciones pasadas y futuras) todo ello del presente mes.

Registro y Login

**Personal Financial Management**
Controla tus finanzas de forma sencilla

[Inicio](#) [Login](#) [Registro](#)

Registro

Email

Password

Login

 **Abraham Quintana Micó** · Full-Stack Developer · Versión 1.0.0

**Personal Financial Management**
Controla tus finanzas de forma sencilla

[Inicio](#) [Login](#) [Registro](#)

Login

Email

Password

Login

 **Abraham Quintana Micó** · Full-Stack Developer · Versión 1.0.0

Perfil de usuario



Personal Financial Management
Controla tus finanzas de forma sencilla

usuario@pfa.com

Inicio

Logout

Perfil

Gestionar categorías

Gestionar transacciones

Estadísticas

Perfil de usuario

Información de la cuenta

Email: usuario@pfa.com

Resumen Financiero

Saldo actual

1189,80 €

Ingresos este mes

1950,33 €

Gastos este mes

760,53 €

Disponible diario

106,74 €

Últimas transacciones registradas

-16.65€	Netflix · 2026-05-02
-125.41€	Supermercado · 2026-05-01
-38.57€	Supermercado · 2026-05-02
-19.50€	Bares · 2026-05-02
-560.40€	Hipoteca · 2026-05-01

 Abraham Quintana Micó · Full-Stack Developer · Versión 1.0.0

Categorías



Personal Financial Management
Controla tus finanzas de forma sencilla

usuario@pfa.com

Inicio

Logout

Perfil

Gestionar categorías

Gestionar transacciones

Estadísticas

Categorías

Atención las categorías no se podrán borrar debido a que están asociadas a transacciones.

Crear

Hipoteca

Supermercado

Seguro

Netflix

Sueldo

Paga extra

Gasolina

Bares

 Abraham Quintana Micó · Full-Stack Developer · Versión 1.0.0

Transacciones



Personal Financial Management
Controla tus finanzas de forma sencilla

usuario@pfa.com

Inicio

Logout

Perfil

Gestionar categorías

Gestionar transacciones

Estadísticas

Transacciones

Crear nueva transacción

Tipo

Gasto

Cantidad

0.00€

Fecha

dd/mm/aaaa

Categoría

-- Selecciona --

Recurrencia

No recurrente

Descripción

Descripción (opcional)

Crear transacción

Listado de transacciones

125.41€ Gasto
Supermercado · 2026-05-01
compra

Eliminar

2200€ Ingreso
Paga extra · 2026-05-15 · Anual
Paga verano

Eliminar

187.46€ Gasto
Seguro · 2026-05-06 · Anual
Seguro coche

Eliminar

38.57€ Gasto
Supermercado · 2026-05-02
Refrescos fiesta

Eliminar

19.5€ Gasto
Bares · 2026-05-02

Eliminar

 Abraham Quintana Micó · Full-Stack Developer · Versión 1.0.0

Estadísticas



Cronograma



Cronograma del Proyecto

Personal Finance Manager | Analista Desarrollador: **Abraham Quintana Micó**

Marzo - Mayo 2026

Entrega: 10/05/2026



Fases Principales
4 Fases



Total Tareas
18 Tareas

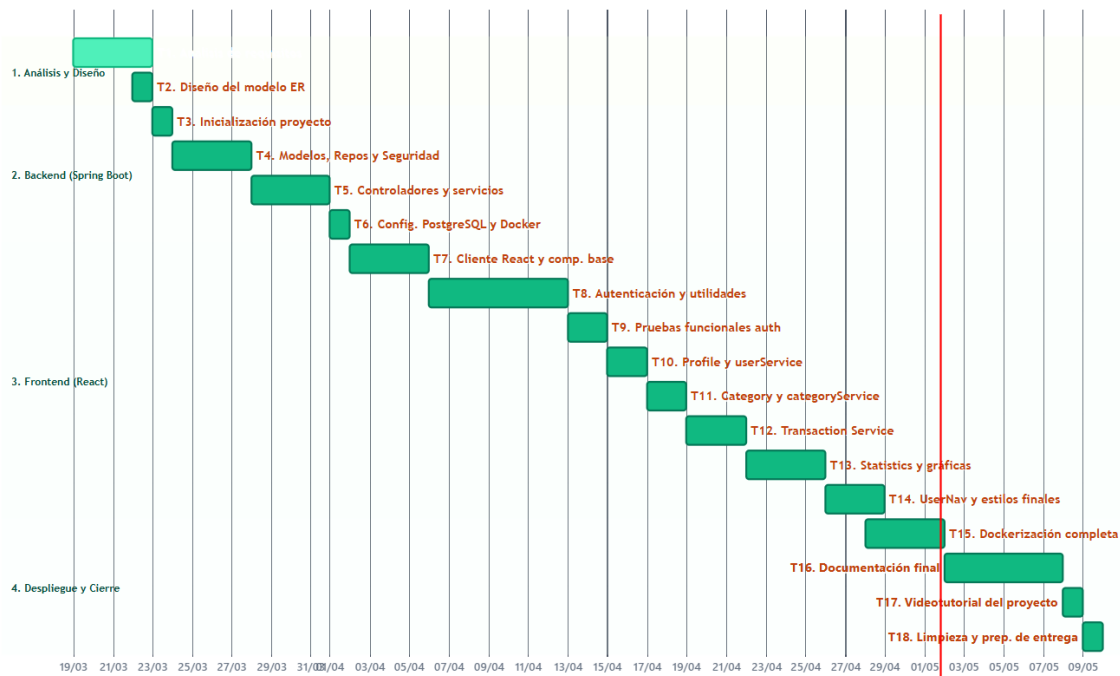


Horas Estimadas
223h



Recursos
1 Desarrollador

Diagrama de Ejecución y Tiempos



Presupuesto

Resumen del Presupuesto:

- Tarifa: 35 €/hora.
- Total horas estimadas: 223.
- Coste recursos humanos: 7.630,00 €.
- Coste recursos materiales estimado: 1.500,00 €.
- Total estimado: 9.130,00 €.

Futuras mejoras

- **Alertas de futuros gastos (seguros, hipotecas, tarjetas, suscripciones...).**
- **Alertas cuando se exceda la media diaria disponible.**
- **Gestión de crédito disponible (tarjetas).**
 - **Cálculo y visualización de intereses mensuales de deudas.**
- **Confirmación manual con pop-up de ingresos y gastos recurrentes antes de incorporarlos al balance.**
- **Gestión de ahorros (huchas y cuentas de ahorro).**
 - **Integración con el balance mensual para cubrir déficits automáticamente.**
 - **Transferencia automática del sobrante mensual a los ahorros.**

- **Registro y seguimiento de inversiones en activos.**
- **Mejoras de UX y mensajes de error más claros en el cliente.**
- **Sistema de roles y panel de administración:**
 - **Gestión de usuarios.**
 - **Reinicio de contraseñas.**
- **Estadísticas de uso y métricas del sistema.**
- **Validación avanzada de contraseñas en el cliente, forzando el uso de contraseñas complejas.**
- **Eliminación de categorías con advertencias sobre transacciones recurrentes asociadas o sistema de desactivación.**
- **Registro de disponibilidad económica para emergencias (amigos y familiares).**



Personal Finance Manager

Aplicación web para gestión personal de finanzas.

Autor: Abraham Quintana Micó